

Construção do Diagrama Esquema Relacional de acordo com as observações das diretrizes da Seção 7 (formas normais e projeto)

Normalização seguida:

Ponto de Partida: Análise dos Dados Brutos

Exemplo de dados de um produto(Todos seguem esse padrão, com exceção dos produtos marcados como “discontinued product”):

padrão:

Id: 1
ASIN: 0827229534
title: Patterns of Preaching: A Sermon Sampler
group: Book
salesrank: 396585
similar: 5 0804215715 156101074X 0687023955 0687074231 082721619X
categories: 2
|Books[283155]|Subjects[1000]|Religion &
Spirituality[22]|Christianity[12290]|Clergy[12360]|Preaching[12368]
|Books[283155]|Subjects[1000]|Religion &
Spirituality[22]|Christianity[12290]|Clergy[12360]|Sermons[12370]
reviews: total: 2 downloaded: 2 avg rating: 5
2000-7-28 cutomer: A2JW67OY8U6HHK rating: 5 votes: 10 helpful: 9
2003-12-14 cutomer: A2VE83MZF98ITY rating: 5 votes: 6 helpful: 5

produtos marcados:

Id: 0
ASIN: 0771044445
discontinued product

Nesta análise inicial, identificamos a entidade **PRODUCT**. Com isso, para garantir que cada produto seja um registro único e inconfundível em nosso sistema, definimos o atributo **Id** como sua Chave Primária. Isso nos deu uma âncora para toda a informação relacionada a ele, onde organizamos os dados da entidade **PRODUCT** em uma tabela só, com todos os atributos aglomerados. Dessa forma, obtivemos o seguinte resultado:

PRODUCT

ID_PRODUCT	ASIN	TITLE	GROUP	SALESRANK	SIMILAR (lista)	CATEGORY (lista)	REVIEW (dados sumários + lista)
------------	------	-------	-------	-----------	--------------------	---------------------	------------------------------------

Tabela PRODUCT (ID_PRODUCT, ASIN, TITLE, GROUP, SALESRANK, SIMILAR (lista), CATEGORY (lista), REVIEWS (dados sumários + lista))

Principais chaves candidatas:

Durante a análise, notamos que tanto o **Id** quanto o **ASIN** são fortes candidatos a identificadores únicos para um produto. Optamos em escolher o **Id** como chave primária para identificação de cada produto, por ser um tipo de dado numérico, o que geralmente oferece melhor performance em junções (joins). O **ASIN** foi mantido com uma restrição de unicidade (UNIQUE constraint) para garantir a integridade dos dados.

Pontos observados:

Após essa primeira análise que usamos como ponto de partida, notamos alguns problemas que consequentemente foram tratados nos próximos passos da normalização, como:

1. Atributos multivariados(SIMILAR, CATEGORY, REVIEW).
2. Necessidade de um view para atributos que correspondem a operações específicas(REVIEW(dados sumários)).

3. Presença de um atributo específico dos dados do sumário da review, que não conseguimos obter através de uma operação como `count(TOTAL_REVIEW)`.

Passo 1: Aplicação da primeira forma normal(1NF)

Conforme a primeira forma normal(1NF), nós quebramos os atributos multivariados obtidos e analisados no passo inicial, em tabelas separadas com seus respectivos atributos. Dessa forma, separamos essa etapa em subetapas para cada atributo multivariado, como **Passo 1 SIMILAR**, **Passo 1 CATEGORY** e **Passo 1 REVIEW**.

Passo 1 Similar:

O atributo similar representa uma lista de produtos similares ao produto que está sendo analisado. Por conta disso, devido o seu sentido sintático e sua estrutura no arquivo principal (`similar: 5 0804215715 156101074X 0687023955 0687074231 082721619X`) consideramos como um atributo multivariado. Contudo, Separamos em uma relação SIMILAR, onde contém apenas o produto analisado(`ID_PRODUCT`) e o atributo que é similar a esse produto(`ASIN_SIMILAR`). Dessa forma, ao analisar os similares de um produto específico, conseguimos obter um conjunto de tuplas contendo todos os produtos similares.

SIMILAR

<u>ID_PRODUCT</u>	<u>ASIN_SIMILAR</u>
-------------------	---------------------

Tabela **SIMILAR** (`ID_PRODUCT`, `ASIN_SIMILAR`)

Principais chaves candidatas:

A chave composta foi escolhida porque ela espelha perfeitamente a regra de negócio. Cada registro em **SIMILAR** representa uma única associação entre um produto de origem e um produto similar. A combinação de **ID_PRODUCT** e **ASIN_SIMILAR** é o que define essa associação de forma inequívoca.

Passo 1 CATEGORY:

O atributo CATEGORY representa um conjunto de categorias em que os produtos podem ser encaixados. Além disso, notamos que um produto pode estar associado a mais de uma categoria, devido a isso nós consideramos o atributo category como um atributo multivariado. Notamos também que esse atributo segue uma espécie de hierarquia entre as categorias, onde o pai sempre de uma categoria é sempre o mesmo, devido a isso decidimos criar duas tabelas: uma tabela CATEGORIES que correlaciona as categorias de acordo com a sua hierarquia, outra que relaciona os produtos às últimas categorias de suas hierarquias.

categories: 2

|Books[283155]|Subjects[1000]|Religion &
Spirituality[22]|Christianity[12290]|Clergy[12360]|Preaching[12368]
|Books[283155]|Subjects[1000]|Religion &
Spirituality[22]|Christianity[12290]|Clergy[12360]|Sermons[12370]

CATEGORIES

<u>ID CATEGORY</u>	CATEGORY_NAME	CATEGORY_FATHER
--------------------	---------------	-----------------

Tabela **CATEGORIES** (**ID_CATEGORY**, **CATEGORY_NAME**, **CATEGORY_FATHER**)

PRODUCT_CATEGORIES

<u>ID PRODUCT</u>	ID_CATEGORY_SON
-------------------	-----------------

Tabela **PRODUCT_CATEGORIES** (**ID_PRODUCT**, **ID_CATEGORY_SON**)

Passo 1 REVIEW:

O atributo REVIEW representa uma combinação de dados sumários (total, downloaded e avg rating) com uma lista de reviews dadas pelos clientes, como visto em:

reviews: total: 2 downloaded: 2 avg rating: 5

2000-7-28 cutomer: A2JW67OY8U6HHK rating: 5 votes: 10 helpful: 9

2003-12-14 cutomer: A2VE83MZF98ITY rating: 5 votes: 6 helpful: 5

Observando isto podemos separar REVIEW em quatro informações distintas, três dessas informações são atômicas (total, downloaded e avg rating) o atributo downloaded sempre é a quantidade de reviews disponíveis no banco de dados, logo não precisa ser guardado, mas há casos em que os valores total e avg rating não estão de acordo com as reviews que podemos acessar, logo estes valores precisam ser guardados em uma tabela, por simplicidade decidimos manter eles na tabela produtos, a última informação do campo é uma lista de reviews deixadas pelos clientes, devido a isso é necessário criar outra tabela para guardar estas reviews.

Exemplo onde a quantidade de reviews total e avg rating não correspondem aos dados disponíveis:

Id: 543142 , ASIN: B000050B7J , Título: Once Upon a Time in China #1, linhas 14621991 até 14621996.

reviews: total: 64 downloaded: 5 avg rating: 4

2004-11-25 cutomer: A32Q2DWQSCJXHR rating: 4 votes: 2 helpful: 0

2005-2-1 cutomer: A1NRCH7YAGZ07B rating: 5 votes: 3 helpful: 3

2005-2-14 cutomer: AXY510Q4IZNP1 rating: 2 votes: 1 helpful: 1

2005-3-9 cutomer: A2BQW2GMEE6BFG rating: 2 votes: 3 helpful: 0

2005-3-22 cutomer: A3AKPY4LXIC24O rating: 4 votes: 4 helpful: 3

Principais chaves candidatas:

A chave composta formada por ID_PRODUCT e CUSTOMER parece promissora à primeira vista, entretanto há exemplos de clientes avaliando o mesmo produto não somente mais de uma vez como também no mesmo dia, por exemplo temos o seguinte caso onde um cliente avaliou o mesmo produto quatro vezes no mesmo dia e outra vez no dia seguinte:

Id: 35063 , ASIN: 0439136369 , Título: Harry Potter and the Prisoner of Azkaban (Book 3) , linhas 955388 até 955392.

1999-9-3	customer: ATVPDKIKX0DER	rating: 5	votes: 1	helpful: 1
1999-9-3	customer: ATVPDKIKX0DER	rating: 5	votes: 0	helpful: 0
1999-9-3	customer: ATVPDKIKX0DER	rating: 5	votes: 0	helpful: 0
1999-9-3	customer: ATVPDKIKX0DER	rating: 5	votes: 0	helpful: 0
1999-9-4	customer: ATVPDKIKX0DER	rating: 5	votes: 0	helpful: 0

Devido a isso foi considerada necessária a criação de um atributo chave ID_REVIEW do tipo SERIAL para diferenciar as avaliações.

REVIEWS

<u>ID REVIEW</u>	ID_PRODUCT	DATE	CUSTOMER	RATING	VOTES	HELPFUL
------------------	------------	------	----------	--------	-------	---------

Tabela REVIEWS (ID_REVIEW, ID_PRODUCT, DATE, CUSTOMER, RATING, VOTES, HELPFUL)

Como um mesmo cliente (customer) pode realizar múltiplas avaliações distribuídas entre um ou mais produtos também decidimos por criar uma tabela CUSTOMERS para armazenar seus IDs, como esta tabela tem um único atributo e ele é único o ID_CUSTOMER foi escolhido como atributo chave:

CUSTOMERS

<u>ID CUSTOMER</u>

Tabela CUSTOMERS(ID_CUSTOMER)

Ao final desta normalização a tabela produtos ficou assim:

PRODUCT

<u>ID PRODUCT</u>	ASIN	TITLE	GROUP	SALESRANK	TOTAL_REVIEWS	AVG_RATING
-------------------	------	-------	-------	-----------	---------------	------------

Tabela PRODUCT (ID_PRODUCT, ASIN, TITLE, GROUP, SALESRANK, TOTAL_REVIEWS, AVG_RATING)

Para satisfazer a forma 2FN o esquema deve obedecer o esquema 1FN e todos os seus atributos não chave devem ser dependentes de todos os componentes do atributo chave de suas tabelas. O esquema apresentado até agora já encaixa-se na 1FN e no último requerimento apontado.

Para satisfazer a forma 3FN o esquema deve obedecer a forma 2FN e não deve conter dependências transitivas, ou seja, situações onde atributos não chave dependem indiretamente de atributos-chave (não há problemas com dependências diretas, como entre ASIN e ID_PRODUCT e vice-versa). O esquema proposto até aqui já encaixa-se nos requerimentos da 3FN.

A Forma Normal de Boyce-Codd (BCFN) segue os mesmos requerimentos da 3FN além de pedir que para cada relação de dependência entre dois atributos na forma $X \rightarrow Y$, X for um atributo capaz de identificar de forma inconfundível cada tupla, ou seja, uma chave primária ou atributo único. Todas as dependências funcionais presentes no esquema já satisfazem a BCFN.

Uma tabela está na forma 4FN se obedece a BCNF e não contém múltiplas dependências independentes entre si com atributos multivariados em relação a um atributo chave, por exemplo, se combinássemos as tabelas SIMILAR e PRODUCT_CATEGORIES teríamos uma tabela onde ASIN_SIMILAR E ID_CATEGORY_SON são atributos multivariados que dependem do ID_PRODUCT, o que violaria o 4FN. O esquema já proposto também já segue a 4FN.

Segue o esquema final do banco de dados:

